Pyroscope

Ai2：Wildfire Risk Assessment Robot

Version Number: 1.0

Instructor: Luyao Niu

Team Members: Annika An, Matthew Taruno, Chenghao Wang

Industrial Sponsor: AI2

2026.3.12

1.Product Overview

Pyroscope is a mobile wildfire risk assessment system designed to help land managers better understand vegetation conditions before fires occur. The system combines a ground-based robotic rover with a web-based dashboard to collect and visualize environmental data across shrubland and wildland areas. Wildfires are strongly influenced by local vegetation conditions such as fuel load, plant temperature, and microclimate. However, most current monitoring methods rely on satellite imagery or aerial surveys, which often lack the fine spatial resolution needed to understand conditions at the ground level. As a result, land managers and researchers often lack reliable data to assess fire risk across specific sites or treatment areas.

Pyroscope addresses this problem by providing **automated ground-level sensing and mapping**. The rover autonomously navigates through a selected survey area and collects environmental measurements using onboard sensors, including a thermal camera, environmental sensors, and imaging systems. These measurements are transmitted to a remote computer where they are processed and stored. The dashboard then visualizes the collected data and generates a fire risk heatmap, allowing users to quickly understand vegetation conditions across the surveyed area.

The system is designed for **land managers, wildfire researchers, and ecological monitoring teams** who need better tools to evaluate vegetation conditions and fire risk in shrubland or wildland environments. By combining autonomous field data collection with interactive visualization, Pyroscope enables more informed decision-making for wildfire prevention and land management.

Primary Features

- **Autonomous Field Survey**
  A tracked rover navigates through a user-defined area and collects environmental measurements at multiple locations.
- **Multi-Sensor Data Collection**
  The system measures vegetation and environmental conditions using a thermal camera, humidity and temperature sensors, and imaging systems.
- **Real-Time Monitoring Dashboard**
  A web-based interface allows users to start missions, monitor rover progress, and view sensor readings in real time.
- **Fire Risk Visualization**
  Collected data are processed into a spatial heatmap where each sampling point is color-coded to indicate estimated wildfire risk.
- **Interactive Data Exploration**
  Users can inspect individual measurement points to view detailed environmental data and better understand vegetation conditions across the area.

2.System Architecture Overview

Pyroscope is a hybrid system with three cooperating runtime layers. The field rover gathers measurements, the backend coordinates data and mission services, and the dashboard presents those results to the operator. This structure lets the project combine robotics autonomy with an easy-to-understand user interface.

2.1 Hardware Layer: Field Rover System

The rover is built on a modified Yahboom Transbot-SE tracked base. It carries a Jetson-class compute module, a LiDAR for navigation awareness, a thermal sensor for temperature-related plant cues, an RGB/depth camera, and a local temperature and humidity sensor. Together, these components allow the rover to move through shrubland-like terrain and record conditions at sampling points.

2.2 Data and Control Layer: Backend System

The backend runs on a remote Ubuntu host and exposes API routes for mission control, scan history, and environmental records. It bridges data from ROS topics into a MySQL database and serves the web dashboard through REST endpoints. The backend also provides the safest central place to check logs when a mission fails or a sensor becomes unavailable.

2.3 User Interaction Layer: Dashboard

The dashboard is the primary user-facing interface. Its design goal is to let a non-technical operator do five things without touching robotics code: open the map, define a scan area, review scan settings, run the mission, and inspect results. The dashboard therefore functions as the mission console, status display, and post-scan review tool.

> **Note:** To explain the product in a non-technical way : the rover collects local vegetation clues in the field, the backend organizes the data, and the dashboard turns those data into a map and a review experience.

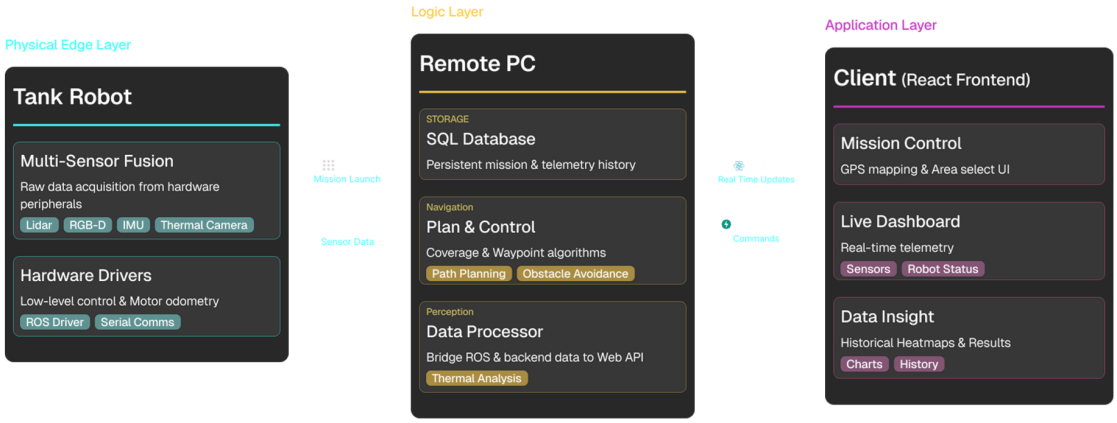


Figure 1. High-level architecture of the Pyroscope rover, backend, and dashboard.

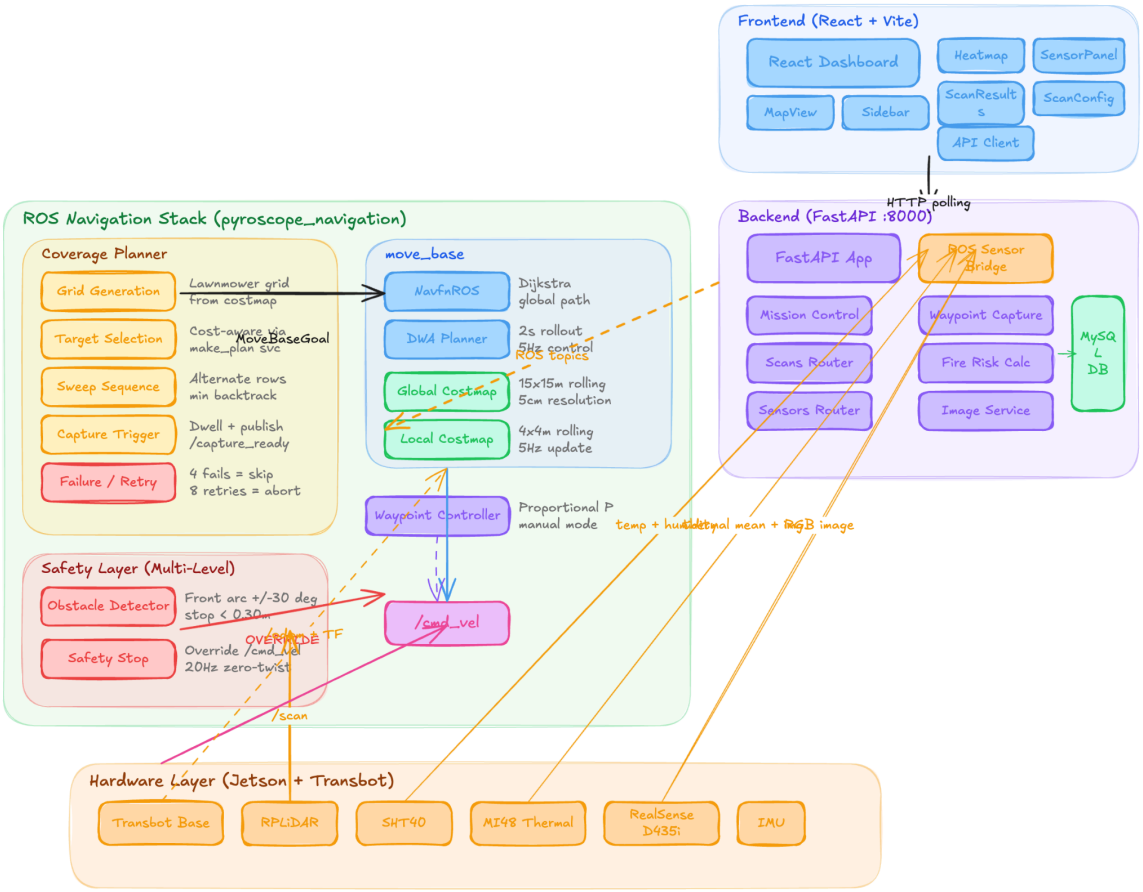


Figure 2. More detailed on diagram particularly on the navigation architecture.

3. Package Contents

Because Pyroscope is a hybrid hardware-software project, package contents include physical robot components, local computing components, and software assets needed to operate the dashboard.

Item	Included	Purpose	Notes
Tracked rover base (Yahboom Transbot-SE)	Yes	Mobility platform	Modified for field sensing
Onboard Jetson-class compute	Yes	Runs robot-side ROS bringup and sensor nodes	Installed on rover
LiDAR sensor	Yes	Obstacle awareness and navigation	Requires unobstructed field of view
Thermal camera / thermal reader	Yes	Plant and ground thermal measurements	May fall back to simulated mode if unavailable
RGB/depth camera	Yes	Scene imaging and navigation context	Mounted on sensor stack
SHT40 temperature / humidity sensor	Yes	Local microclimate sensing	Connected through Arduino or serial pipeline
Battery pack and charger	Yes	Power for field operations	Use approved charger only
Remote host laptop or PC	Operator supplied	Runs backend, planning, and dashboard services	Ubuntu recommended for full stack
Dashboard frontend code	Yes	Mission UI and result review	React + Vite
Backend code and database migrations	Yes	API services and persistence	FastAPI + MySQL
Cables, mounts, and field fasteners	Yes	Assembly and maintenance	Check before every run

4. System Requirements

Hybrid systems must satisfy both hardware and software requirements. The tables below separate operator-facing requirements from development or deployment requirements.

4.1 Software and computing requirements

Category	Requirement	Why it matters
Supported operating system	Ubuntu 18.04 for the full ROS stack; a modern browser can be used for dashboard viewing once services are running	ROS Melodic and the provided launch flow are built for Ubuntu 18.04
Host PC	8 GB RAM minimum; recommended multi-core CPU; Wi-Fi or Ethernet on the same local network as the robot	Runs backend services and can also host mission planning and dashboard
Robot computer	Jetson Nano or similar Jetson-class onboard computer	Runs robot-side ROS bringup and sensor processes
Backend dependencies	Python 3.9+, pip, virtual environment support, FastAPI dependencies, Uvicorn	Needed for API server and sensor bridge
Frontend dependencies	Node.js 18+ and npm	Needed to build and run the dashboard
Database	MySQL 8+ and a local MySQL user account	Stores scan records, environmental data, images, and status information
Network requirement	Robot and host machine must be on the same flat local network; internet is not required during field operation	ROS multi-machine communication is sensitive to network configuration
User accounts	Local OS account; MySQL user; optional backend operator credentials if authentication routes are enabled	Setup and persistence require at least a database user

4.2 Hardware and field requirements

Category	Requirement	Why it matters
Power	Battery must start at a high charge level; use the manufacturer-approved charger; do not run while charging	Low voltage causes alarms, unstable motion, and sensor dropouts
Environmental conditions	Dry outdoor operation only; avoid rain, mist, standing water, and high dust when possible	The system has no waterproofing or IP rating
Terrain compatibility	Best on dry to moderately firm shrubland-like terrain with low obstacles	Loose gravel, deep sand, dense brush, and steep slopes reduce reliability
Sensor compatibility	Thermal camera, LiDAR, and cameras must have clear lines of sight after any mechanical change	Blocked sensors produce poor navigation or unusable data
Required tools	Laptop, spare USB cables, charger, zip ties or Velcro, basic hand tools, multimeter for power checks	Needed for setup, recovery, and field maintenance

5. Installation and Setup

This section is written as a first-time setup procedure. A new operator should follow the steps in order. If the system has already been deployed once, Sections 5.1 and 5.2 can be reused as a pre-run checklist before each field session.

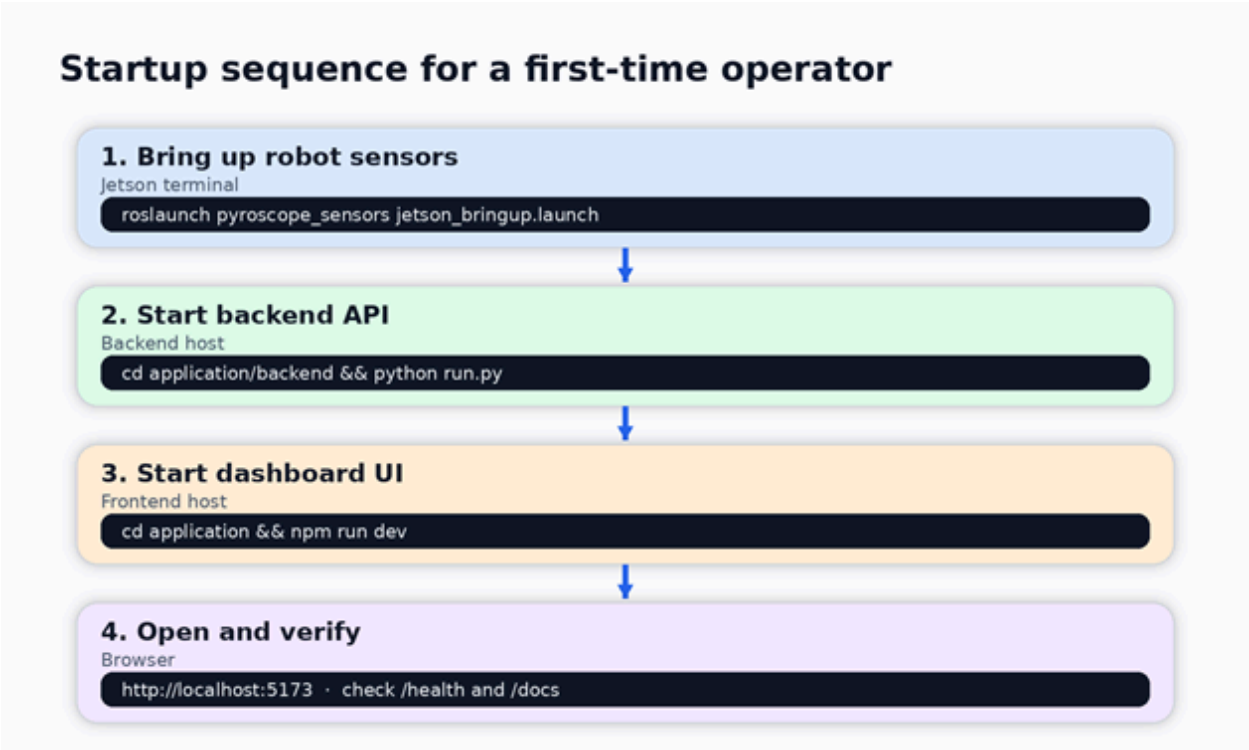


Figure 1. Recommended startup sequence for a first-time operator.

5.1 Before the first deployment

1. Prepare two machines: the robot-side Jetson and a host computer for backend, planning, and dashboard services.

2. Place both machines on the same local network and record the Jetson IP address and host PC IP address.

3. Inspect the rover mechanically: verify tight sensor mounts, secure cables, and a charged battery.

4. Confirm that the LiDAR and cameras are not blocked by the shell, mast, or loose wiring.

5.2 Hardware preparation

1. Power off the rover before touching mounts or cables.

2. Connect the battery and listen for abnormal alarms. A continuous bi bi bi warning indicates low voltage or a power issue.

3. Check that the thermal camera, RGB/depth camera, LiDAR, and SHT40 sensor are physically connected.

4. Place the rover on flat ground for the initial test so the first teleoperation and sensor checks happen in a safe environment.

5.3 Build the ROS workspace

1. On the Jetson and on any Ubuntu host that will run ROS navigation, open a terminal and go to the catkin workspace.

2. Build the workspace with catkin_make.

3. Source the workspace with source devel/setup.bash before launching robot or navigation nodes.

```
cd ~/pyroscope/catkin_ws
 catkin_make
 source devel/setup.bash
```

5.4 Set up the backend service

1. On the host computer, navigate to application/backend.

2. Create and activate a Python virtual environment.

3. Install required packages from requirements.txt.

4. Create a MySQL database named pyroscope_db, then create or confirm a MySQL user with permission to access it.

5. Create or edit the .env file to include DATABASE_URL, JWT_SECRET_KEY, ROS_MASTER_URI, and ROS_IP.

6. Run database migrations with alembic upgrade head.

```
cd ~/pyroscope/application/backend
 python -m venv venv
 source venv/bin/activate
 pip install -r requirements.txt
 alembic upgrade head
```

```
DATABASE_URL=mysql+pymysql://<user>:<password>@localhost:3306/pyroscope_db
 JWT_SECRET_KEY=<your-secret>
 ROS_MASTER_URI=http://<JETSON_IP>:11311
 ROS_IP=<BACKEND_PC_IP>
```

5.5 Set up the dashboard frontend

1. On the machine that will host the web UI, go to the application directory.

2. Install Node.js dependencies with npm install.

3. Check the API base URL in application/src/services/api.js and update it if the backend is not running on the default host or port.

4. Start the development server with npm run dev.

```
cd ~/pyroscope/application
 npm install
 npm run dev
```

Note: If the dashboard opens but shows no data, the most common first-time issue is that the frontend is still pointing at the wrong backend address in api.js.

5.6 First-time startup verification

1. On the Jetson, launch the robot bringup and sensor stack.

2. On the backend host, start the backend server.

3. On the frontend host, start the dashboard and open http://localhost:5173 in a browser.

4. Verify that backend health and API docs are reachable, then confirm that live sensor values begin to populate.

Machine	Launch command
Jetson	roslaunch pyroscope_sensors jetson_bringup.launch
Backend host	cd ~/pyroscope/application/backend && python3 run.py
Frontend host	cd ~/pyroscope/application && npm run dev

6. Operating Instructions

This section explains how to operate the system after setup. It is intentionally written for non-technical users first, then followed by the exact start, stop, and shutdown actions required for a safe run.

6.1 What the dashboard is for

The dashboard is the operator control panel. It is where the user sets the scan area, reviews mission parameters, starts the mission, watches the rover progress, and checks the final results. For most demo and field-review workflows, users should be able to complete the mission from the dashboard without interacting with robotics code directly.

6.2 Dashboard areas explained in plain language

Dashboard area	What the user sees	What it means
Map and mission panel	A map with the target area and mission controls	Used to define where the rover should scan
Scan settings	Fields such as area size, row spacing, waypoint spacing, dwell time, or timeout	Controls how dense, how slow, and how long the scan will be
Live status cards	Mission progress, current phase, and sensor snapshots	Confirms whether the robot is actively collecting data
Heatmap and point results	Color-coded locations and drill-down details	Shows where the system estimated relatively higher or lower local fire risk
Scan history and data log	Previous records from the database	Lets the user revisit completed scans and compare outputs

6.3 How a non-technical user should run one mission

1. Open the dashboard in the browser and wait for the map and prior scan records to load.

2. Select or draw the area of interest on the map. Keep the area within the practical operating boundary of the rover.

3. Review the scan configuration. In simple terms: row spacing decides how far apart each pass is, waypoint spacing decides how often the rover samples, dwell time decides how long it pauses to measure, and timeout is the safety limit for mission completion.

4. Click Start Scan. The rover should begin moving through the area in a systematic pattern.

5. Watch the progress indicators and live sensor snapshots. If the rover stops, stalls, or shows missing sensors, pause the session and diagnose the issue before continuing.

6. When the scan is complete, review the heatmap and inspect individual points to see detailed measurements such as temperature, humidity, and fuel-related indicators.

7. Record conclusions carefully. The map should be used as a screening and comparison tool, not as the sole basis for a burn authorization or a safety-critical decision.

6.4 Recommended explanation to non-technical stakeholders

When presenting the dashboard during a review, a clear script is: First, we choose the scan area on the map. Then we adjust a few scan settings if needed. During the mission, we watch the rover status and live sensor readings. When the mission is complete, the dashboard shows a fire-risk heatmap and lets us click each point for more detailed measurements.

6.5 Start, stop, and emergency controls

Action	How to do it	When to use it
Start mission	Use the Start Scan control in the dashboard after setting the area and parameters	Normal mission launch
Stop mission from software	POST to /robot/mission/stop or use the dashboard stop control if exposed	Controlled stop from the UI or API
Immediate velocity stop	Publish one zero-velocity command to /cmd_vel	Emergency rover halt
Manual teleoperation backup	roslaunch transbot_ctrl transbot_keyboard.launch	When autonomous behavior needs human intervention

```
rostopic pub /cmd_vel geometry_msgs/Twist '{linear: {x: 0.0}, angular: {z: 0.0}}' --once
```

6.6 Proper shutdown procedure

1. Stop the mission and confirm that the rover has come to rest.

2. Close the dashboard browser tab after data have finished saving.

3. Use Ctrl+C to stop the frontend, backend, and ROS launch processes in a controlled order.

4. Power down the rover after software processes have been stopped.

5. Disconnect or charge the battery only after the rover is fully powered down.

7. Technical Specifications

7.1 Software architecture and technologies

Layer	Technology and integration	Purpose
Frontend	React 18 + Vite + Leaflet / React-Leaflet	Mission UI, map interaction, data display
Backend	FastAPI + Uvicorn	REST API, mission services, data handling
Database	MySQL 8+ with SQLAlchemy ORM and Alembic migrations	Persistence for scan records, environmental data, images, and status
Robot communication	ROS Melodic multi-machine network	Sensor topics, launch orchestration, navigation control
Navigation stack	move_base, NavfnROS, DWA local planner, EKF odometry	Coverage motion and local obstacle avoidance
Media and uploads	Image upload routes and stored scan imagery	Associates observations with scan records

7.2 Database and API notes

The backend project structure uses models, schemas, routers, services, and utility modules. API endpoints in the current repository include authentication routes, scan list and detail routes, environmental upload routes, and image upload routes. This makes the backend both an operator service and a development interface for future dashboard features.

Representative API or data object	Purpose
/api/scans	List previous scans and retrieve scan details
/api/environmental	Store or upload environmental measurements
/api/images/upload	Upload or associate mission images
scan_records	Stores mission-level records
environmental_data	Stores point-level environmental measurements
robot status records	Support mission and availability feedback

7.3 Hardware specifications

Component	Specification
Rover base	Modified Yahboom Transbot-SE tracked platform
Maximum nominal motion	Approx. 0.4 m/s linear, 2.0 rad/s angular
Thermal sensor	MI48-class thermal capture path, 80 x 62 px, about 0.33 Hz
RGB/depth camera	Intel RealSense D435i, 640 x 480 at 6 FPS in current documentation
LiDAR	2D RPLidar publishing to /scan
Temperature and humidity sensor	SHT40 via Arduino Nano or serial bridge, about 1 Hz

Onboard compute	Jetson Nano or similar Jetson-class module
Power source	Battery-powered tracked rover platform

8. Safety and Operational Precautions

Because Pyroscope includes moving hardware, batteries, exposed sensors, and local data services, safe operation depends on both physical precautions and disciplined software use.

8.1 Electrical safety

• Use only the approved charger for the rover battery pack.

• Do not operate the rover while charging.

• Stop immediately if you smell overheating electronics, see battery swelling, or hear continuous undervoltage alarms.

• Low battery can produce erratic motor behavior, unexpected restarts, or dropped sensor streams.

8.2 Mechanical safety

• Tracks, gears, and moving parts can pinch fingers, cables, and clothing.

• Keep hands clear of the drivetrain when the robot is powered.

• Raised sensors and a shifted center of mass increase tipping risk on obstacles or slopes.

• Power off before adjusting mounts, routing cables, or checking connectors.

8.3 Environmental safety

• The system is intended for dry shrubland-like terrain and is not waterproof.

• Do not operate in rain, mist, standing water, or heavy mud.

• Avoid dense vegetation that can snag cables or fully occlude sensors.

• Maintain line of sight to the rover during autonomous runs.

8.4 Safe interpretation of results

• Dashboard heatmaps should be treated as field intelligence and screening support, not as a certified prediction of fire spread or burn safety.

• Operators should combine Pyroscope output with site knowledge, weather forecasts, and human judgment before making management decisions.

• Do not disable safety stop behaviors in field deployments simply to complete a run faster.

8.5 Data privacy and security considerations

• The backend stores scan records, environmental data, and uploaded images locally in MySQL and the uploads directory.

• If the authentication routes are used, protect JWT secrets and do not commit real secrets to version control.

• Because the frontend can point to a configurable API host, verify that it is connected only to a trusted backend on the project network.

9. Troubleshooting

The following issues are the ones most likely to affect a first-time operator. The dashboard symptoms are listed first because they are usually what the user sees before they know whether the failure is in networking, sensors, the backend, or the rover.

Symptom	Likely cause	Recommended action
Dashboard loads but shows no live data	Frontend is pointing at the wrong API host or backend is not running	Verify api.js base URL, confirm backend is running on port 8000, and check /docs or health route
Dashboard shows sensors unavailable	One or more ROS topics are not publishing	Check thermal, temperature, humidity, and camera topics; restart the missing sensor node
Mission will not start	MySQL is not running, migrations were skipped, or mission services are unavailable	Confirm database access, run alembic upgrade head, and check backend logs
Rover does not move after Start Scan	ROS_MASTER_URI or network settings are incorrect, or navigation stack is not active	Verify Jetson and host IP settings, same-network connectivity, and move_base or coverage launch status
Thermal camera not found	MI48 device or library is unavailable	Use simulated thermal mode for development and reconnect the real device for field use

Scan record appears but point data are incomplete	Known waypoint capture reliability issue or interrupted mission	Restart backend, review service logs, and repeat a short test mission before a full deployment
Robot emits continuous bi bi bi alarm	Battery undervoltage or power-board issue	Stop immediately, power down if needed, and recharge or inspect the power system
Map opens but area selection feels wrong	Operator selected an area beyond the safe field boundary or the map scale is misunderstood	Reduce the scan area and verify the intended boundary before relaunching

10. Limitations

- There needs to be tuning of the parameters of the coverage planner to ensure reliability.
- The dashboard currently relies on polling behavior rather than a full low-latency streaming architecture, so updates are not instantaneous.
- The multi-machine ROS setup is sensitive to local networking. Firewalls, NAT, or inconsistent IP settings can break communication.
- The wildfire-risk output is a relative local assessment aid. It is not a substitute for burn plans, agency procedures, or expert review.
- The current system is most appropriate for shrubland and prescribed-burn style survey units rather than all wildfire environments.
- Thermal sensing may fall back to simulated values when the physical thermal stack or supporting library is unavailable.
- Odometry-based positioning can drift over longer missions if GPS or other global corrections are not integrated in the active deployment.
- Waypoint data capture and some result-record persistence paths are still under active refinement.
- The frontend currently assumes a configured backend endpoint and may require manual adjustment in api.js when the deployment host changes.

Appendix A. Quick Start Checklist

• Battery charged; no undervoltage alarm.

• Sensors mounted securely and not blocked.

• Jetson and host PC on the same network.

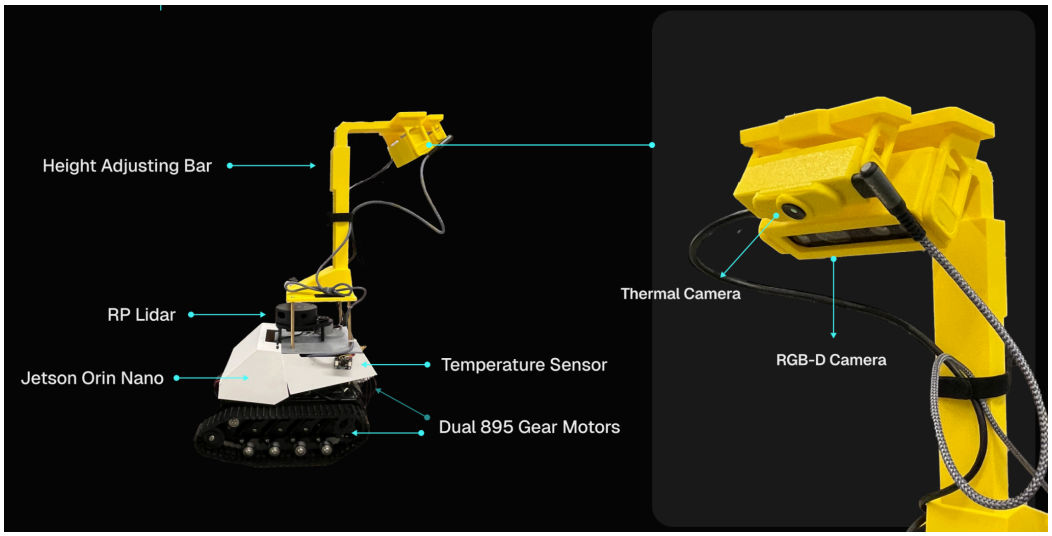
• MySQL running; backend .env configured.

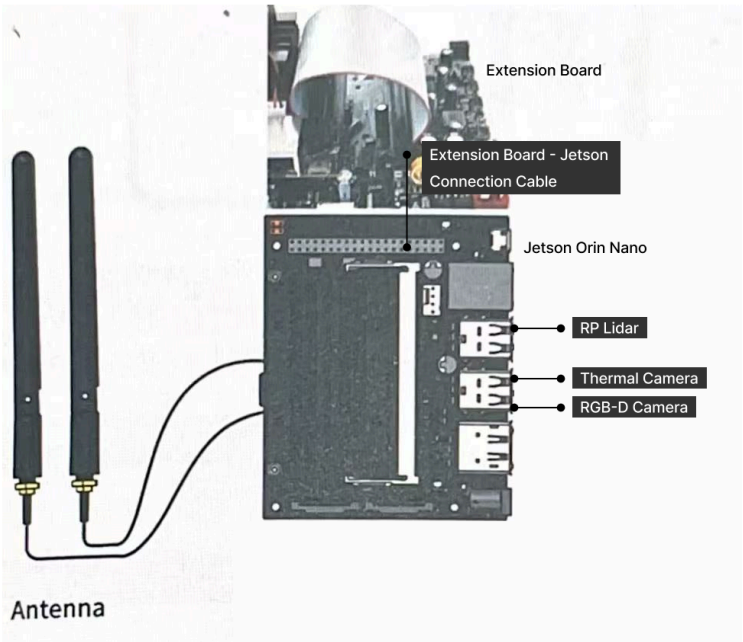
• Frontend API base URL verified.

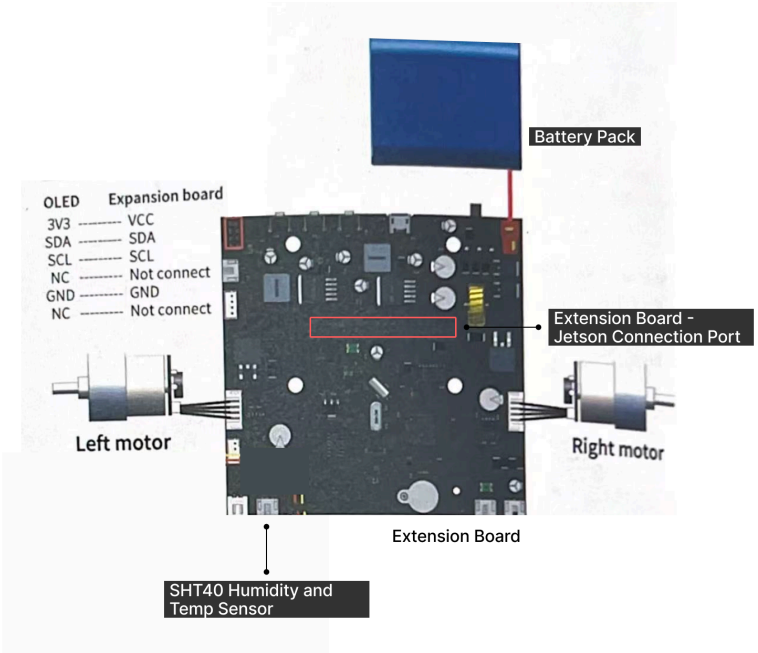
• ROS bringup active; backend active; dashboard open.

• Short teleop or smoke test completed before full mission.

Appendix B. Key Components & Connection Map




Extension Board
Extension Board - Jetson Connection Cable
Jetson Orin Nano
RP Lidar
Thermal Camera
RGB-D Camera
Antenna


Battery Pack
OLED
Expansion board
3V3 -------- VCC
SDA -------- SDA
SCL -------- SCL
NC -------- Not connect
GND -------- GND
NC -------- Not connect
Extension Board - Jetson Connection Port
Left motor
Right motor
Extension Board
SHT40 Humidity and Temp Sensor